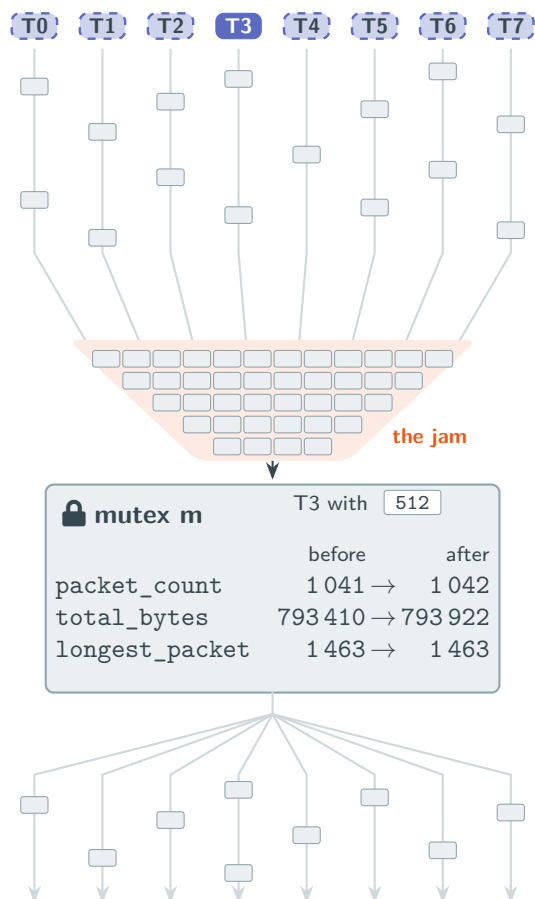


Task 3: Statistics in a 5G base station

A 5G base station receives millions of packets per second from the phones around it, and eight threads process them. The station keeps three statistics: how many packets have come so far, how many bytes they had together (for monitoring and billing), and the length of the longest packet so far (to size the buffers). The code is correct, but the profiler shows that the threads spend most of their time waiting for one mutex. Make it faster.

Note: this is a very simplified model, made for teaching.

The situation



1 Loose

Eight threads, each with its own packets. Every thread takes a packet and calls `on_packet(len)`. Plenty of room: the threads work at the same time.

2 The jam

All of them want the same mutex. Only one can go in; the other seven wait in front of the door. More threads only make the jam longer.

3 One at a time

Inside the mutex, T3 updates the statistics with its packet of 512 bytes, and gets the number 1 041 back.

4 Loose again

After the mutex every thread goes on alone with its packet: no more waiting.

8 threads, but at the mutex only 1 of them works at a time

The code

The template is in [Task3](#). This is stats.cpp:

```
// To check that your solution is correct, change TEST from 0 to 1.
const int TEST = 0;

#include <stdint>
#include <mutex>
#include "packets.h"    // ready: the speed measurement and the tests

std::mutex m;
uint32_t packet_count  = 0; // how many packets so far; the next packet gets this number
uint64_t total_bytes   = 0; // the length of all packets together
uint32_t longest_packet = 0; // the length of the longest packet so far

// called for every packet; returns its sequence number
uint32_t on_packet(uint32_t len) {
    std::lock_guard<std::mutex> lock(m);
    uint32_t number = packet_count++;
    total_bytes += len;
    if (len > longest_packet) {
        longest_packet = len;
    }
    return number;
}

int main() {
    // TEST 0: measure the speed, TEST 1: run the tests
    return run(TEST, total_bytes, longest_packet);
}
```

In the task folder, make run compile the program and runs it.

The program measures the speed: 8 threads send 1 000 000 packets each, and it prints how many million packets per second on_packet() handled.

Your tasks

- ❶ Run it. How many packets per second does the version with the mutex handle?
- ❷ Replace the mutex with std::atomic. To check your solution, change TEST from 0 to 1 at the top of stats.cpp: the program then does not measure, but runs ready tests. Do all tests pass? If not, why not, when every variable is atomic?
- ❸ Set TEST back to 0. How many times faster is it now?

The tests (TEST 1)

They are ready in packets.h; you do not write them. 8 threads send packets, and the program checks four things: the sequence numbers are unique, the byte counter is exact, longest_packet never goes down, and longest_packet is right at the end.

Atomic variables

`std::atomic<int>` (from `<atomic>`) is an `int` whose every operation happens in **one step**: no other thread can come in the middle of it. In the same way `std::atomic<uint32_t>` is an atomic `uint32_t`.

```
std::atomic<int> x{5};           // declare: an atomic int that starts at 5

int old = x.fetch_add(3);        // add 3, return the value before: x = 8, old = 5
x++;                             // the same as x.fetch_add(1): x = 9

int now = x.load();              // read: now = 9
if (x > 7) { /* ... */ }         // a plain read works too, it is the same as x.load() > 7

x = 10;                          // write: x = 10
```